JavaScript Complete Reference Guide

Comprehensive JavaScript Training Manual

Confide Computer Academy © COPYRIGHT 2025

CONFIDE COMPUTER ACADEMY

© COPYRIGHT 2025

Table of Contents

2

1. Introduction to JavaScript 6
    - What is JavaScript? 6
    - Adding JavaScript to HTML 6
    - Console Methods 7
    - Comments 7
2. Variables and Data Types 8
    - Variable Declaration 8
    - Variable Naming Rules 8
    - Data Types 8
    - Type Checking 9
    - Type Conversion 10
3. Operators 11
    - Arithmetic Operators 11
    - Assignment Operators 11
    - Comparison Operators 11
    - Logical Operators 12
    - Ternary Operator 12
    - Nullish Coalescing (??) 12
    - Optional Chaining (?.) 13
4. Control Flow 14
    - If Statements 14
    - Switch Statement 14
    - For Loop 15
    - While Loop 16
    - For...of Loop (Arrays) 16
    - For...in Loop (Objects) 16
    - Break and Continue 17
5. Functions 18
    - Function Declaration 18
    - Function Expression 18
    - Arrow Functions 18
    - Parameters and Arguments 18
    - Return Statement 19
    - Callback Functions 19
    - Higher-Order Functions 20
    - IIFE (Immediately Invoked Function Expression) 20
    - Scope and Closures 20
6. Arrays 22

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Creating Arrays 22
Accessing Elements 22
Array Properties 22
Adding/Removing Elements 23
Array Methods 23
Spread Operator with Arrays 25
Destructuring Arrays 26
7. Objects 27
Creating Objects 27
Accessing Properties 27
Adding/Modifying Properties 28
Methods 28
The 'this' Keyword 28
Object Methods 29
Object Destructuring 29
Spread Operator with Objects 30
Computed Property Names 31
JSON 31
8. DOM Manipulation 32
Selecting Elements 32
Modifying Content 32
Modifying Attributes 32
Modifying Styles 33
Working with Classes 33
Creating and Inserting Elements 34
Removing Elements 35
Cloning Elements 35
Traversing DOM 35
9. Events 37
Adding Event Listeners 37
Removing Event Listeners 37
Common Event Types 37
Event Object 38
Event Methods 39
Event Bubbling and Capturing 39
Event Delegation 40
Form Handling 40
10. Asynchronous JavaScript 42
setTimeout and setInterval 42
Callbacks 42
Promises 43

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Async/Await 44
Fetch API 45
11. Local Storage 48
localStorage 48
Storing Objects 48
sessionStorage 49
Storage Events 49
Practical Examples 49
12. ES6+ Features 51
Template Literals 51
Destructuring 51
Spread Operator 51
Rest Parameters 51
Enhanced Object Literals 52
Modules 52
Classes 53
Symbols 54
Iterators and Generators 55
13. Error Handling 57
try...catch...finally 57
Throwing Errors 57
Custom Errors 57
Error Object Properties 58
Async Error Handling 58
14. JavaScript Best Practices 59
Code Organization 59
Function Best Practices 59
Variable Declarations 60
Comparison 60
Error Handling 61
Async Operations 61
Performance 61
Comments 62
15. JavaScript Cheat Sheet 64
Variables 64
Data Types 64
Operators 64
Functions 64
Arrays 64
Objects 64
DOM 64

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Async 65
ES6+ 65
Resources 65

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

1. Introduction to JavaScript

What is JavaScript?

JavaScript is a high-level, interpreted programming language that enables interactive web pages. It's the programming layer of web technologies, alongside HTML (structure) and CSS (presentation).

Key Characteristics:

- **Client-side**: Runs in the browser
- **Dynamic**: Can change content in real-time
- **Event-driven**: Responds to user interactions
- **Versatile**: Also used for server-side (Node.js), mobile apps, desktop apps

Adding JavaScript to HTML

External JavaScript (Recommended)

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <h1>Hello World</h1>

    <!-- Place before closing body tag -->
    <script src="script.js"></script>
</body>
</html>
```

Internal JavaScript

```
<script>
    console.log("Hello World");
    alert("Welcome!");
</script>
```

Inline JavaScript (Avoid)

```
<button onclick="alert('Clicked!')">Click Me</button>
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Console Methods

```
console.log("Normal message");
console.info("Information message");
console.warn("Warning message");
console.error("Error message");
console.table([{name: "John", age: 30}]);
console.clear(); // Clear console
console.time("timer"); // Start timer
console.timeEnd("timer"); // End timer
```

Comments

```
// Single line comment

/*
Multi-line
comment
*/

/**
 * Documentation comment
 * @param {string} name - User name
 * @returns {string} Greeting message
 */
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

2. Variables and Data Types

Variable Declaration

```
// var (old way - avoid)
var oldVariable = "Don't use var";

// let (can be reassigned)
let userName = "John";
userName = "Jane"; // OK

// const (cannot be reassigned)
const PI = 3.14159;
// PI = 3.14; // Error!

// Multiple declarations
let x = 5, y = 10, z = 15;
```

Variable Naming Rules

```
// Valid names
let userName;
let _private;
let $element;
let user2;
let camelCaseVariable;

// Invalid names
// let 2user; // Can't start with number
// let user-name; // No hyphens
// let let; // Can't use reserved words
```

Data Types

Primitive Types:

```
// String
let name = "John";
let message = 'Hello';
let template = `Hello ${name}`;

// Number
let age = 30;
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
let price = 19.99;
let negative = -5;
let infinity = Infinity;
let notANumber = NaN;

// Boolean
let isActive = true;
let isCompleted = false;

// Undefined
let x;
console.log(x); // undefined

// Null
let empty = null;

// Symbol (unique identifier)
let id = Symbol("id");

// BigInt (large integers)
let bigNumber = 12345678901234567890123456789012345678901234567890n;
```

Reference Types:

```
// Object
let person = {
    name: "John",
    age: 30
};

// Array
let colors = ["red", "green", "blue"];

// Function
function greet() {
    return "Hello";
}

// Date
let today = new Date();
```

Type Checking

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
typeof "Hello"; // "string"
typeof 42; // "number"
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object" (known bug)
typeof {}; // "object"
typeof []; // "object"
typeof function(){}; // "function"

// Better array check
Array.isArray([]); // true

// Check for null
let value = null;
value === null; // true
```

Type Conversion

```
// To String
String(123); // "123"
(123).toString(); // "123"
123 + ""; // "123"

// To Number
Number("123"); // 123
parseInt("123"); // 123
parseFloat("123.45"); // 123.45
+"123"; // 123

// To Boolean
Boolean(1); // true
Boolean(0); // false
Boolean(""); // false
Boolean("text"); // true
!!"text"; // true
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

3. Operators

Arithmetic Operators

```
let a = 10, b = 3;

// Basic operations
a + b; // 13 (addition)
a - b; // 7 (subtraction)
a * b; // 30 (multiplication)
a / b; // 3.333... (division)
a % b; // 1 (modulus/remainder)
a ** b; // 1000 (exponentiation)

// Increment/Decrement
let x = 5;
x++; // x = 6 (post-increment)
++x; // x = 7 (pre-increment)
x--; // x = 6 (post-decrement)
--x; // x = 5 (pre-decrement)
```

Assignment Operators

```
let x = 10;

x = 5;    // x = 5
x += 3;   // x = x + 3 → x = 8
x -= 2;   // x = x - 2 → x = 6
x *= 4;   // x = x * 4 → x = 24
x /= 3;   // x = x / 3 → x = 8
x %= 5;   // x = x % 5 → x = 3
x **= 2;  // x = x ** 2 → x = 9
```

Comparison Operators

```
// Equality
5 == "5";   // true (loose equality - converts types)
5 === "5";  // false (strict equality - checks type)
5 != "5";   // false
5 !== "5";  // true

// Relational
5 > 3;      // true
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
5 < 3;      // false
5 >= 5;     // true
5 <= 4;     // false

// Special cases
null == undefined;   // true
null === undefined;  // false
NaN === NaN;        // false (use isNaN())
```

Logical Operators

```
// AND (&&) - both must be true
true && true;   // true
true && false;  // false

// OR (||) - at least one must be true
true || false;  // true
false || false; // false

// NOT (!) - inverts boolean
!true;   // false
!false;  // true

// Short-circuit evaluation
let result = condition && doSomething(); // doSomething() only if condition is true
let value = userInput || "default"; // use default if userInput is falsy
```

Ternary Operator

```
// condition ? valueIfTrue : valueIfFalse
let age = 18;
let status = age >= 18 ? "adult" : "minor";

// Nested ternary
let grade = score >= 90 ? "A" :
            score >= 80 ? "B" :
            score >= 70 ? "C" : "F";
```

Nullish Coalescing (??)

```
// Returns right side if left is null or undefined
let value = null ?? "default"; // "default"
let value2 = 0 ?? "default"; // 0 (not null/undefined)
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// vs OR operator
let value3 = 0 || "default"; // "default" (0 is falsy)
```

Optional Chaining (?.)

```
let user = {
    name: "John",
    address: {
        street: "Main St"
    }
};

// Without optional chaining
let zipCode = user && user.address && user.address.zipCode;

// With optional chaining
let zipCode = user?.address?.zipCode; // undefined (no error)

// With arrays
let firstItem = array?.[0];

// With functions
let result = obj.method?.();
```

Confide Computer Academy ©COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

4. Control Flow

If Statements

```
// Basic if
if (condition) {
    // code
}

// If-else
if (condition) {
    // code if true
} else {
    // code if false
}

// If-else if-else
if (condition1) {
    // code
} else if (condition2) {
    // code
} else if (condition3) {
    // code
} else {
    // code
}

// Example
let age = 18;
if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

Switch Statement

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    case value3:
    case value4:
        // code for both value3 and value4
        break;
    default:
        // code if no match
}

// Example
let day = 2;
switch (day) {
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    case 6:
    case 7:
        console.log("Weekend");
        break;
    default:
        console.log("Other day");
}
```

For Loop

```
// Standard for loop
for (let i = 0; i < 5; i++) {
    console.log(i); // 0, 1, 2, 3, 4
}

// Multiple variables
for (let i = 0, j = 10; i < 5; i++, j--) {
    console.log(i, j);
}

// Infinite loop (careful!)
for (;;) {
    // runs forever
    break; // need break to exit
}
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

While Loop

```
// While loop
let i = 0;
while (i < 5) {
    console.log(i);
    i++;
}

// Do-while (runs at least once)
let j = 0;
do {
    console.log(j);
    j++;
} while (j < 5);
```

For...of Loop (Arrays)

```
let fruits = ["apple", "banana", "orange"];

for (let fruit of fruits) {
    console.log(fruit);
}

// With index
for (let [index, fruit] of fruits.entries()) {
    console.log(index, fruit);
}
```

For...in Loop (Objects)

```
let person = {
    name: "John",
    age: 30,
    city: "New York"
};

for (let key in person) {
    console.log(key, person[key]);
}

// Note: for...in also works with arrays but not recommended
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Break and Continue

```
// Break - exit loop
for (let i = 0; i < 10; i++) {
    if (i === 5) break;
    console.log(i); // 0, 1, 2, 3, 4
}

// Continue - skip iteration
for (let i = 0; i < 5; i++) {
    if (i === 2) continue;
    console.log(i); // 0, 1, 3, 4
}

// Labels (rarely used)
outer: for (let i = 0; i < 3; i++) {
    for (let j = 0; j < 3; j++) {
        if (i === 1 && j === 1) break outer;
        console.log(i, j);
    }
}
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

5. Functions

Function Declaration

```
function greet(name) {
    return `Hello, ${name}!`;
}

let message = greet("John"); // "Hello, John!"
```

Function Expression

```
const greet = function(name) {
    return `Hello, ${name}!`;
};
```

Arrow Functions

```
// Basic syntax
const greet = (name) => {
    return `Hello, ${name}!`;
};

// Concise syntax (implicit return)
const greet = name => `Hello, ${name}!`;

// Multiple parameters
const add = (a, b) => a + b;

// No parameters
const sayHello = () => "Hello!";

// Object return (needs parentheses)
const makeObject = (name) => ({ name: name });
```

Parameters and Arguments

```
// Default parameters
function greet(name = "Guest") {
    return `Hello, ${name}!`;
}
greet(); // "Hello, Guest!"
greet("John"); // "Hello, John!"
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Rest parameters
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}
sum(1, 2, 3, 4, 5); // 15

// Destructuring parameters
function printUser({ name, age }) {
    console.log(`${name} is ${age} years old`);
}
printUser({ name: "John", age: 30 });
```

Return Statement

```
// Return value
function add(a, b) {
    return a + b;
}

// Return early
function divide(a, b) {
    if (b === 0) {
        return "Cannot divide by zero";
    }
    return a / b;
}

// No return (returns undefined)
function logMessage(msg) {
    console.log(msg);
}
```

Callback Functions

```
// Function as parameter
function processUser(name, callback) {
    console.log(`Processing ${name}...`);
    callback();
}

processUser("John", function() {
    console.log("Done!");
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
});

// Common with array methods
let numbers = [1, 2, 3, 4, 5];
numbers.forEach(function(num) {
    console.log(num);
});
```

Higher-Order Functions

```
// Function returning function
function multiplier(factor) {
    return function(number) {
        return number * factor;
    };
}

let double = multiplier(2);
let triple = multiplier(3);

console.log(double(5)); // 10
console.log(triple(5)); // 15
```

IIFE (Immediately Invoked Function Expression)

```
// Runs immediately
(function() {
    console.log("This runs immediately!");
})();

// With parameters
(function(name) {
    console.log(`Hello, ${name}!`);
})("John");

// Arrow IIFE
(() => {
    console.log("Arrow IIFE");
})();
```

Scope and Closures

```
// Function scope
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
function outer() {
    let outerVar = "I'm outside";

    function inner() {
        let innerVar = "I'm inside";
        console.log(outerVar); // Accessible
    }

    inner();
    // console.log(innerVar); // Error - not accessible
}

// Closure
function counter() {
    let count = 0;

    return {
        increment: function() {
            count++;
            return count;
        },
        decrement: function() {
            count--;
            return count;
        },
        getCount: function() {
            return count;
        }
    };
}

let myCounter = counter();
myCounter.increment(); // 1
myCounter.increment(); // 2
myCounter.getCount(); // 2
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

6. Arrays

Creating Arrays

```
// Array literal
let fruits = ["apple", "banana", "orange"];

// Array constructor
let numbers = new Array(1, 2, 3, 4, 5);

// Empty array
let empty = [];

// Array with length
let arr = new Array(5); // [empty × 5]

// Mixed types
let mixed = [1, "two", true, null, { name: "John" }];
```

Accessing Elements

```
let fruits = ["apple", "banana", "orange"];

// Index (starts at 0)
fruits[0]; // "apple"
fruits[1]; // "banana"

// Last element
fruits[fruits.length - 1]; // "orange"

// Using at() method
fruits.at(0); // "apple"
fruits.at(-1); // "orange" (negative index from end)
```

Array Properties

```
let arr = [1, 2, 3, 4, 5];

arr.length; // 5

// Change length
arr.length = 3; // [1, 2, 3]
arr.length = 5; // [1, 2, 3, empty × 2]
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Adding/Removing Elements

```
let fruits = ["apple", "banana"];

// Add to end
fruits.push("orange"); // ["apple", "banana", "orange"]
fruits.push("grape", "mango"); // Multiple items

// Remove from end
let last = fruits.pop(); // "mango"

// Add to beginning
fruits.unshift("strawberry"); // ["strawberry", "apple", "banana", "orange", "grape"]

// Remove from beginning
let first = fruits.shift(); // "strawberry"

// Add/Remove at specific position
// splice(start, deleteCount, item1, item2, ...)
fruits.splice(1, 1, "kiwi"); // Remove 1 at index 1, add "kiwi"
fruits.splice(2, 0, "melon"); // Add "melon" at index 2 (remove 0)
```

Array Methods

Iteration Methods:

```
let numbers = [1, 2, 3, 4, 5];

// forEach - iterate
numbers.forEach((num, index) => {
    console.log(index, num);
});

// map - transform array
let doubled = numbers.map(num => num * 2);
// [2, 4, 6, 8, 10]

// filter - filter array
let evens = numbers.filter(num => num % 2 === 0);
// [2, 4]

// find - find first match
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
let found = numbers.find(num => num > 3);
// 4

// findIndex - find index
let index = numbers.findIndex(num => num > 3);
// 3

// some - test if any match
let hasEven = numbers.some(num => num % 2 === 0);
// true

// every - test if all match
let allPositive = numbers.every(num => num > 0);
// true

// reduce - reduce to single value
let sum = numbers.reduce((total, num) => total + num, 0);
// 15

// reduceRight - reduce from right
let result = numbers.reduceRight((acc, num) => acc - num);
```

Search Methods:

```
let fruits = ["apple", "banana", "orange", "banana"];

// indexOf
fruits.indexOf("banana"); // 1
fruits.indexOf("grape"); // -1 (not found)

// lastIndexOf
fruits.lastIndexOf("banana"); // 3

// includes
fruits.includes("apple"); // true
fruits.includes("grape"); // false
```

Other Methods:

```
let arr = [1, 2, 3, 4, 5];

// join - array to string
arr.join(", "); // "1, 2, 3, 4, 5"
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// slice - extract portion (doesn't modify original)
arr.slice(1, 3); // [2, 3]
arr.slice(-2); // [4, 5]

// concat - merge arrays
let arr2 = [6, 7];
arr.concat(arr2); // [1, 2, 3, 4, 5, 6, 7]

// reverse - reverse array (modifies original)
arr.reverse(); // [5, 4, 3, 2, 1]

// sort - sort array (modifies original)
arr.sort(); // Alphabetical sort
arr.sort((a, b) => a - b); // Numeric ascending
arr.sort((a, b) => b - a); // Numeric descending

// flat - flatten nested arrays
let nested = [1, [2, 3], [4, [5, 6]]];
nested.flat(); // [1, 2, 3, 4, [5, 6]]
nested.flat(2); // [1, 2, 3, 4, 5, 6] (depth 2)

// flatMap - map then flat
[1, 2, 3].flatMap(x => [x, x * 2]); // [1, 2, 2, 4, 3, 6]

// fill - fill with value
arr.fill(0); // [0, 0, 0, 0, 0]
arr.fill(1, 2, 4); // Fill 1 from index 2 to 4

// from - create array from iterable
Array.from("hello"); // ["h", "e", "l", "l", "o"]
Array.from({length: 5}, (_, i) => i); // [0, 1, 2, 3, 4]

// isArray - check if array
Array.isArray([]); // true
Array.isArray({}); // false
```

Spread Operator with Arrays

```
let arr1 = [1, 2, 3];
let arr2 = [4, 5, 6];

// Copy array
let copy = [...arr1]; // [1, 2, 3]
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Merge arrays
let merged = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

// Add elements
let extended = [0, ...arr1, 4]; // [0, 1, 2, 3, 4]

// Function arguments
Math.max(...arr1); // 3
```

Destructuring Arrays

```
let arr = [1, 2, 3, 4, 5];

// Basic destructuring
let [first, second] = arr; // first = 1, second = 2

// Skip elements
let [first, , third] = arr; // first = 1, third = 3

// Rest elements
let [first, ...rest] = arr; // first = 1, rest = [2, 3, 4, 5]

// Default values
let [a, b, c = 0] = [1, 2]; // a = 1, b = 2, c = 0

// Swapping variables
let x = 1, y = 2;
[x, y] = [y, x]; // x = 2, y = 1
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

7. Objects

Creating Objects

```
// Object literal
let person = {
    name: "John",
    age: 30,
    city: "New York"
};

// Object constructor
let person = new Object();
person.name = "John";
person.age = 30;

// Constructor function
function Person(name, age) {
    this.name = name;
    this.age = age;
}
let john = new Person("John", 30);
```

Accessing Properties

```
let person = {
    name: "John",
    age: 30,
    "favorite color": "blue"
};

// Dot notation
person.name; // "John"
person.age; // 30

// Bracket notation
person["name"]; // "John"
person["favorite color"]; // "blue"

// Dynamic property
let prop = "age";
person[prop]; // 30
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Adding/Modifying Properties

```
let person = {
    name: "John"
};

// Add property
person.age = 30;
person["city"] = "New York";

// Modify property
person.name = "Jane";

// Delete property
delete person.age;
```

Methods

```
let person = {
    name: "John",
    age: 30,

    // Method
    greet: function() {
        return `Hello, I'm ${this.name}`;
    },

    // Shorthand method syntax
    introduce() {
        return `I'm ${this.name}, ${this.age} years old`;
    }
};

person.greet(); // "Hello, I'm John"
person.introduce(); // "I'm John, 30 years old"
```

The 'this' Keyword

```
let person = {
    name: "John",
    greet: function() {
        console.log(this.name); // 'this' refers to person object
    },
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    // Arrow function doesn't bind 'this'
    arrowGreet: () => {
        console.log(this.name); // 'this' refers to outer scope
    }
};

person.greet(); // "John"
```

Object Methods

```
let person = {
    name: "John",
    age: 30,
    city: "New York"
};

// Object.keys - array of keys
Object.keys(person); // ["name", "age", "city"]

// Object.values - array of values
Object.values(person); // ["John", 30, "New York"]

// Object.entries - array of [key, value] pairs
Object.entries(person);
// [["name", "John"], ["age", 30], ["city", "New York"]]

// Object.assign - copy properties
let copy = Object.assign({}, person);
let merged = Object.assign({}, person, { country: "USA" });

// Object.freeze - prevent modifications
Object.freeze(person);
person.name = "Jane"; // Ignored (no error in non-strict mode)

// Object.seal - prevent adding/deleting properties
Object.seal(person);
person.email = "john@example.com"; // Ignored

// hasOwnProperty - check if property exists
person.hasOwnProperty("name"); // true
person.hasOwnProperty("email"); // false
```

Object Destructuring

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
let person = {
    name: "John",
    age: 30,
    city: "New York"
};

// Basic destructuring
let { name, age } = person;
// name = "John", age = 30

// Different variable names
let { name: userName, age: userAge } = person;
// userName = "John", userAge = 30

// Default values
let { name, country = "USA" } = person;
// name = "John", country = "USA"

// Rest properties
let { name, ...rest } = person;
// name = "John", rest = { age: 30, city: "New York" }

// Nested destructuring
let user = {
    name: "John",
    address: {
        city: "New York",
        zip: "10001"
    }
};
let { address: { city, zip } } = user;
```

Spread Operator with Objects

```
let person = {
    name: "John",
    age: 30
};

// Copy object
let copy = { ...person };

// Merge objects
let additional = { city: "New York", country: "USA" };
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
let merged = { ...person, ...additional };

// Override properties
let updated = { ...person, age: 31 };
```

Computed Property Names

```
let propName = "age";

let person = {
    name: "John",
    [propName]: 30, // age: 30
    ["favorite" + "Color"]: "blue" // favoriteColor: "blue"
};
```

JSON

```
let person = {
    name: "John",
    age: 30,
    hobbies: ["reading", "coding"]
};

// Object to JSON string
let json = JSON.stringify(person);
// '{"name":"John","age":30,"hobbies":["reading","coding"]}'

// JSON string to object
let parsed = JSON.parse(json);
// { name: "John", age: 30, hobbies: ["reading", "coding"] }

// Pretty print
let prettyJson = JSON.stringify(person, null, 2);
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

8. DOM Manipulation

Selecting Elements

```
// By ID
let element = document.getElementById("myId");

// By class name (returns HTMLCollection)
let elements = document.getElementsByClassName("myClass");

// By tag name
let paragraphs = document.getElementsByTagName("p");

// Query selector (first match)
let element = document.querySelector(".myClass");
let element = document.querySelector("#myId");
let element = document.querySelector("div.container > p");

// Query selector all (returns NodeList)
let elements = document.querySelectorAll(".myClass");
let elements = document.querySelectorAll("li");
```

Modifying Content

```
let element = document.querySelector("#myElement");

// innerHTML - HTML content
element.innerHTML = "<p>New HTML content</p>";

// textContent - text only
element.textContent = "Plain text";

// innerText - visible text only
element.innerText = "Visible text";

// value - for form inputs
let input = document.querySelector("input");
input.value = "New value";
```

Modifying Attributes

```
let link = document.querySelector("a");
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Get attribute
link.getAttribute("href");

// Set attribute
link.setAttribute("href", "https://example.com");
link.setAttribute("target", "_blank");

// Remove attribute
link.removeAttribute("target");

// hasAttribute
link.hasAttribute("href"); // true

// Direct property access
link.href = "https://example.com";
link.id = "myLink";
link.className = "active";

// Dataset (data-* attributes)
// <div data-user-id="123" data-role="admin"></div>
let div = document.querySelector("div");
div.dataset.userId; // "123"
div.dataset.role; // "admin"
div.dataset.newProp = "value"; // data-new-prop="value"
```

Modifying Styles

```
let element = document.querySelector("#myElement");

// Individual styles
element.style.color = "red";
element.style.backgroundColor = "blue";
element.style.fontSize = "20px";
element.style.display = "none";

// Multiple styles
element.style.cssText = "color: red; background: blue; font-size: 20px;";

// Get computed style
let styles = window.getComputedStyle(element);
styles.color; // "rgb(255, 0, 0)"
```

Working with Classes

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
let element = document.querySelector("#myElement");

// Add class
element.classList.add("active");
element.classList.add("class1", "class2", "class3");

// Remove class
element.classList.remove("inactive");

// Toggle class
element.classList.toggle("visible"); // Add if not present, remove if present

// Check if has class
element.classList.contains("active"); // true/false

// Replace class
element.classList.replace("old-class", "new-class");

// Class name (string)
element.className = "class1 class2 class3";
```

Creating and Inserting Elements

```
// Create element
let div = document.createElement("div");
let p = document.createElement("p");
let text = document.createTextNode("Hello");

// Set content/attributes
div.textContent = "Hello World";
div.id = "newDiv";
div.classList.add("container");

// Append (to end)
document.body.appendChild(div);
parent.appendChild(child);

// Insert before
parent.insertBefore(newElement, referenceElement);

// Insert adjacent
element.insertAdjacentHTML("beforebegin", "<p>Before</p>");
// Positions: beforebegin, afterbegin, beforeend, afterend
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Modern methods
parent.append(child1, child2, "text"); // Multiple items
parent.prepend(child); // To beginning
element.before(newElement); // Before element
element.after(newElement); // After element
```

Removing Elements

```
let element = document.querySelector("#myElement");

// Remove element
element.remove();

// Remove child
parent.removeChild(child);

// Remove all children
element.innerHTML = "";
// or
while (element.firstChild) {
    element.removeChild(element.firstChild);
}
```

Cloning Elements

```
let element = document.querySelector("#myElement");

// Clone element
let clone = element.cloneNode(false); // Shallow clone (no children)
let deepClone = element.cloneNode(true); // Deep clone (with children)

document.body.appendChild(clone);
```

Traversing DOM

```
let element = document.querySelector("#myElement");

// Parent
element.parentNode;
element.parentElement;

// Children
element.children; // HTMLCollection (only elements)
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
element.childNodes; // NodeList (all nodes including text)
element.firstElementChild;
element.lastElementChild;
element.childElementCount;

// Siblings
element.nextElementSibling;
element.previousElementSibling;

// Closest ancestor matching selector
element.closest(".container");
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

9. Events

Adding Event Listeners

```
let button = document.querySelector("button");

// addEventListener
button.addEventListener("click", function(event) {
    console.log("Button clicked!", event);
});

// Arrow function
button.addEventListener("click", (e) => {
    console.log("Clicked!");
});

// Named function
function handleClick(e) {
    console.log("Clicked!");
}
button.addEventListener("click", handleClick);

// Multiple listeners
button.addEventListener("click", handler1);
button.addEventListener("click", handler2);
```

Removing Event Listeners

```
function handleClick(e) {
    console.log("Clicked!");
}

// Add listener
button.addEventListener("click", handleClick);

// Remove listener (must be same function)
button.removeEventListener("click", handleClick);
```

Common Event Types

Mouse Events:

```
element.addEventListener("click", handler);
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Confide Computer Academy © COPYRIGHT 2025

```
element.addEventListener("dblclick", handler);
element.addEventListener("mousedown", handler);
element.addEventListener("mouseup", handler);
element.addEventListener("mouseenter", handler); // No bubbling
element.addEventListener("mouseleave", handler); // No bubbling
element.addEventListener("mouseover", handler); // With bubbling
element.addEventListener("mouseout", handler); // With bubbling
element.addEventListener("mousemove", handler);
element.addEventListener("contextmenu", handler); // Right click
```

Keyboard Events:

```
element.addEventListener("keydown", handler);
element.addEventListener("keyup", handler);
element.addEventListener("keypress", handler); // Deprecated

// Get key information
document.addEventListener("keydown", (e) => {
    console.log(e.key); // "a", "Enter", "ArrowUp"
    console.log(e.code); // "KeyA", "Enter", "ArrowUp"
    console.log(e.keyCode); // Numeric code (deprecated)
});
```

Form Events:

```
form.addEventListener("submit", handler);
input.addEventListener("change", handler); // After value changes
input.addEventListener("input", handler); // During typing
input.addEventListener("focus", handler);
input.addEventListener("blur", handler);
```

Window/Document Events:

```
window.addEventListener("load", handler); // Page fully loaded
window.addEventListener("DOMContentLoaded", handler); // DOM ready
window.addEventListener("resize", handler);
window.addEventListener("scroll", handler);
window.addEventListener("beforeunload", handler); // Before leaving page
```

Event Object

```
element.addEventListener("click", (e) => {
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    // Target
    e.target; // Element that triggered event
    e.currentTarget; // Element with listener attached

    // Position (mouse events)
    e.clientX, e.clientY; // Relative to viewport
    e.pageX, e.pageY; // Relative to page
    e.offsetX, e.offsetY; // Relative to element

    // Keys (keyboard events)
    e.key; // Key value
    e.code; // Physical key
    e.shiftKey; // true if shift pressed
    e.ctrlKey; // true if ctrl pressed
    e.altKey; // true if alt pressed
    e.metaKey; // true if cmd/win pressed

    // Type
    e.type; // Event type ("click", "keydown", etc.)

    // Timestamp
    e.timeStamp; // Time when event occurred
});
```

Event Methods

```
element.addEventListener("click", (e) => {
    // Prevent default behavior
    e.preventDefault(); // Prevent form submission, link navigation, etc.

    // Stop propagation
    e.stopPropagation(); // Stop event from bubbling up
    e.stopImmediatePropagation(); // Stop other listeners on same element
});
```

Event Bubbling and Capturing

```
// Bubbling (default) - from target to root
element.addEventListener("click", handler, false);
element.addEventListener("click", handler); // Same as above

// Capturing - from root to target
element.addEventListener("click", handler, true);
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Example
parent.addEventListener("click", () => {
    console.log("Parent");
});

child.addEventListener("click", (e) => {
    console.log("Child");
    // e.stopPropagation(); // Uncomment to stop bubbling
});

// Clicking child logs: "Child" then "Parent"
```

Event Delegation

```
// Instead of adding listener to each item
document.querySelector("ul").addEventListener("click", (e) => {
    if (e.target.tagName === "LI") {
        console.log("List item clicked:", e.target.textContent);
    }
});

// Example: Delete button in dynamically added items
document.querySelector("#container").addEventListener("click", (e) => {
    if (e.target.classList.contains("delete-btn")) {
        e.target.parentElement.remove();
    }
});
```

Form Handling

```
let form = document.querySelector("form");

form.addEventListener("submit", (e) => {
    e.preventDefault(); // Prevent page reload

    // Get form data
    let formData = new FormData(form);

    // Get individual values
    let name = formData.get("name");
    let email = formData.get("email");
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    // Or access inputs directly
    let name = form.elements.name.value;
    let email = form.elements.email.value;

    // Validation
    if (!name || !email) {
        alert("Please fill all fields");
        return;
    }

    // Process data
    console.log({ name, email });
});
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

10. Asynchronous JavaScript

setTimeout and setInterval

```
// setTimeout - run once after delay
setTimeout(() => {
    console.log("Runs after 2 seconds");
}, 2000);

// With arguments
setTimeout((name) => {
    console.log(`Hello, ${name}`);
}, 1000, "John");

// Cancel timeout
let timeoutId = setTimeout(() => {
    console.log("This won't run");
}, 3000);
clearTimeout(timeoutId);

// setInterval - run repeatedly
let intervalId = setInterval(() => {
    console.log("Runs every second");
}, 1000);

// Cancel interval
clearInterval(intervalId);

// Example: Countdown
let count = 10;
let countdown = setInterval(() => {
    console.log(count);
    count--;
    if (count < 0) {
        clearInterval(countdown);
        console.log("Done!");
    }
}, 1000);
```

Callbacks

```
// Callback function
function fetchData(callback) {
    setTimeout(() => {
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
        let data = { name: "John", age: 30 };
        callback(data);
    }, 1000);
}

fetchData((data) => {
    console.log(data);
});

// Callback hell (avoid!)
getData((data) => {
    processData(data, (result) => {
        saveData(result, (response) => {
            console.log(response);
        });
    });
});
```

Promises

```
// Creating a Promise
let promise = new Promise((resolve, reject) => {
    setTimeout(() => {
        let success = true;
        if (success) {
            resolve("Success!");
        } else {
            reject("Error!");
        }
    }, 1000);
});

// Consuming Promise
promise
    .then((result) => {
        console.log(result); // "Success!"
        return result.toUpperCase();
    })
    .then((upper) => {
        console.log(upper); // "SUCCESS!"
    })
    .catch((error) => {
        console.error(error);
    })
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    .finally(() => {
        console.log("Always runs");
    });

// Promise methods
Promise.resolve(42); // Immediately resolved promise
Promise.reject("Error"); // Immediately rejected promise

// Promise.all - wait for all
Promise.all([promise1, promise2, promise3])
    .then(([result1, result2, result3]) => {
        console.log(result1, result2, result3);
    });

// Promise.race - first to complete
Promise.race([promise1, promise2])
    .then((result) => {
        console.log("First:", result);
    });

// Promise.allSettled - wait for all (doesn't fail)
Promise.allSettled([promise1, promise2])
    .then((results) => {
        results.forEach((result) => {
            if (result.status === "fulfilled") {
                console.log("Success:", result.value);
            } else {
                console.log("Failed:", result.reason);
            }
        });
    });
```

Async/Await

```
// Async function
async function fetchData() {
    return "Data"; // Automatically wrapped in Promise
}

// Await (only in async functions)
async function getData() {
    try {
        let result = await fetchData();
        console.log(result);
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    } catch (error) {
        console.error(error);
    }
}

// Multiple awaits
async function processData() {
    try {
        let data1 = await fetchData1();
        let data2 = await fetchData2(data1);
        let data3 = await fetchData3(data2);
        return data3;
    } catch (error) {
        console.error("Error:", error);
    }
}

// Parallel execution
async function getMultiple() {
    let [data1, data2, data3] = await Promise.all([
        fetchData1(),
        fetchData2(),
        fetchData3()
    ]);
    return { data1, data2, data3 };
}
```

Fetch API

```
// GET request
fetch("https://api.example.com/data")
    .then((response) => {
        if (!response.ok) {
            throw new Error("Network response was not ok");
        }
        return response.json();
    })
    .then((data) => {
        console.log(data);
    })
    .catch((error) => {
        console.error("Error:", error);
    });
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// With async/await
async function fetchData() {
    try {
        let response = await fetch("https://api.example.com/data");

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.error("Error:", error);
    }
}

// POST request
async function postData() {
    try {
        let response = await fetch("https://api.example.com/data", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                name: "John",
                age: 30
            })
        });

        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.error("Error:", error);
    }
}

// Other HTTP methods
fetch(url, {
    method: "PUT", // or DELETE, PATCH, etc.
    headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer token"
    },
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    body: JSON.stringify(data)
});
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

11. Local Storage

localStorage

```
// Set item
localStorage.setItem("name", "John");
localStorage.setItem("age", "30");

// Get item
let name = localStorage.getItem("name"); // "John"

// Remove item
localStorage.removeItem("name");

// Clear all
localStorage.clear();

// Check if key exists
if (localStorage.getItem("name")) {
    console.log("Name exists");
}

// Get number of items
let count = localStorage.length;

// Get key by index
let key = localStorage.key(0);

// Iterate through all items
for (let i = 0; i < localStorage.length; i++) {
    let key = localStorage.key(i);
    let value = localStorage.getItem(key);
    console.log(key, value);
}
```

Storing Objects

```
// Store object (must stringify)
let user = {
    name: "John",
    age: 30,
    hobbies: ["reading", "coding"]
};
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
localStorage.setItem("user", JSON.stringify(user));

// Retrieve object (must parse)
let storedUser = JSON.parse(localStorage.getItem("user"));
console.log(storedUser.name); // "John"

// Check before parsing
let userData = localStorage.getItem("user");
if (userData) {
    let user = JSON.parse(userData);
    console.log(user);
}
```

sessionStorage

```
// Same API as localStorage
// Data cleared when tab/window closes

sessionStorage.setItem("temp", "value");
let temp = sessionStorage.getItem("temp");
sessionStorage.removeItem("temp");
sessionStorage.clear();
```

Storage Events

```
// Listen for storage changes (works across tabs)
window.addEventListener("storage", (e) => {
    console.log("Key:", e.key);
    console.log("Old value:", e.oldValue);
    console.log("New value:", e.newValue);
    console.log("URL:", e.url);
    console.log("Storage:", e.storageArea);
});
```

Practical Examples

Save Form Data:

```
// Save on input
document.querySelector("input").addEventListener("input", (e) => {
    localStorage.setItem("draft", e.target.value);
});
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Load on page load
window.addEventListener("DOMContentLoaded", () => {
    let draft = localStorage.getItem("draft");
    if (draft) {
        document.querySelector("input").value = draft;
    }
});
```

Theme Preference:

```
// Save theme
function setTheme(theme) {
    document.body.className = theme;
    localStorage.setItem("theme", theme);
}

// Load theme
let theme = localStorage.getItem("theme") || "light";
document.body.className = theme;
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

12. ES6+ Features

Template Literals

```
let name = "John";
let age = 30;

// Template literal
let message = `Hello, my name is ${name} and I'm ${age} years old.`;

// Multi-line
let html = `
    <div>
        <h1>${name}</h1>
        <p>Age: ${age}</p>
    </div>
`;

// Expression
let total = `Total: ${10 + 20}`;
```

Destructuring

Already covered in Arrays and Objects sections.

Spread Operator

```
// Arrays
let arr1 = [1, 2, 3];
let arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

// Objects
let obj1 = { a: 1, b: 2 };
let obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// Function arguments
Math.max(...[1, 5, 3, 9, 2]); // 9
```

Rest Parameters

```
function sum(...numbers) {
    return numbers.reduce((total, num) => total + num, 0);
}
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
sum(1, 2, 3, 4, 5); // 15
```

Enhanced Object Literals

```
let name = "John";
let age = 30;

let person = {
    // Property shorthand
    name, // name: name
    age, // age: age

    // Method shorthand
    greet() { // greet: function() {
        return `Hello, ${this.name}`;
    },

    // Computed property names
    ["prop" + "Name"]: "value"
};
```

Modules

Exporting:

```
// math.js
export const PI = 3.14159;

export function add(a, b) {
    return a + b;
}

export function subtract(a, b) {
    return a - b;
}

// Default export
export default class Calculator {
    // class definition
}
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Importing:

```
// Import named exports
import { PI, add, subtract } from "./math.js";

// Import default
import Calculator from "./math.js";

// Import all
import * as Math from "./math.js";
// Math.PI, Math.add()

// Import with alias
import { add as addition } from "./math.js";

// Import default and named
import Calculator, { PI, add } from "./math.js";
```

Classes

```
// Class declaration
class Person {
    // Constructor
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }

    // Method
    greet() {
        return `Hello, I'm ${this.name}`;
    }

    // Getter
    get info() {
        return `${this.name}, ${this.age}`;
    }

    // Setter
    set age(value) {
        if (value < 0) {
            throw new Error("Age cannot be negative");
        }
        this._age = value;
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    }

    // Static method
    static species() {
        return "Homo sapiens";
    }
}

// Create instance
let john = new Person("John", 30);
john.greet(); // "Hello, I'm John"
Person.species(); // "Homo sapiens"

// Inheritance
class Student extends Person {
    constructor(name, age, grade) {
        super(name, age); // Call parent constructor
        this.grade = grade;
    }

    // Override method
    greet() {
        return `${super.greet()}. I'm a student.`;
    }
}

let jane = new Student("Jane", 20, "A");
```

Symbols

```
// Create symbol
let id = Symbol("id");
let id2 = Symbol("id");

id === id2; // false (always unique)

// Use as object property
let user = {
    name: "John",
    [id]: 123
};

// Hidden from normal iteration
Object.keys(user); // ["name"]
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Confide Computer Academy © COPYRIGHT 2025

```
Object.getOwnPropertySymbols(user); // [Symbol(id)]
```

Iterators and Generators

```
// Iterator
let range = {
    from: 1,
    to: 5,

    [Symbol.iterator]() {
        return {
            current: this.from,
            last: this.to,

            next() {
                if (this.current <= this.last) {
                    return { done: false, value: this.current++ };
                } else {
                    return { done: true };
                }
            }
        };
    }
};

for (let num of range) {
    console.log(num); // 1, 2, 3, 4, 5
}

// Generator function
function* generateSequence() {
    yield 1;
    yield 2;
    yield 3;
}

let generator = generateSequence();
generator.next(); // { value: 1, done: false }
generator.next(); // { value: 2, done: false }
generator.next(); // { value: 3, done: false }
generator.next(); // { value: undefined, done: true }

// Use in for...of
for (let value of generateSequence()) {
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    console.log(value); // 1, 2, 3
}
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

13. Error Handling

try...catch...finally

```
try {
    // Code that might throw error
    let result = riskyOperation();
    console.log(result);
} catch (error) {
    // Handle error
    console.error("Error occurred:", error.message);
} finally {
    // Always executes
    console.log("Cleanup");
}
```

Throwing Errors

```
// Throw error
throw new Error("Something went wrong");

// Throw with specific error type
throw new TypeError("Expected a number");
throw new RangeError("Value out of range");
throw new ReferenceError("Variable not defined");

// Throw custom value
throw "Custom error message";
throw { code: 404, message: "Not found" };
```

Custom Errors

```
class ValidationError extends Error {
    constructor(message) {
        super(message);
        this.name = "ValidationError";
    }
}

function validateAge(age) {
    if (age < 0) {
        throw new ValidationError("Age cannot be negative");
    }
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
    if (age > 120) {
        throw new ValidationError("Age is too high");
    }
}

try {
    validateAge(-5);
} catch (error) {
    if (error instanceof ValidationError) {
        console.log("Validation error:", error.message);
    } else {
        throw error; // Rethrow if not validation error
    }
}
```

Error Object Properties

```
try {
    throw new Error("Something went wrong");
} catch (error) {
    console.log(error.name); // "Error"
    console.log(error.message); // "Something went wrong"
    console.log(error.stack); // Stack trace
}
```

Async Error Handling

```
// With promises
fetch(url)
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error("Error:", error));

// With async/await
async function fetchData() {
    try {
        let response = await fetch(url);
        let data = await response.json();
        console.log(data);
    } catch (error) {
        console.error("Error:", error);
    }
}
```

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

14. JavaScript Best Practices

Code Organization

✅ **DO:**

```
// Use meaningful names
let userName = "John";
let userAge = 30;

// Use constants for fixed values
const MAX_ITEMS = 100;
const API_URL = "https://api.example.com";

// One declaration per line
let firstName = "John";
let lastName = "Doe";

// Use camelCase for variables/functions
let myVariable = 10;
function calculateTotal() {}

// Use PascalCase for classes
class UserProfile {}
```

❌ **DON'T:**

```
// Cryptic names
let x = "John";
let n = 30;

// Multiple declarations on one line
let a = 1, b = 2, c = 3;
```

Function Best Practices

✅ **DO:**

```
// Keep functions small and focused
function calculateTax(amount) {
    return amount * 0.1;
}
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Use arrow functions for callbacks
numbers.map(n => n * 2);

// Return early to avoid nesting
function processUser(user) {
    if (!user) return;
    if (!user.active) return;

    // Process user
}
```

Variable Declarations

✅ **DO:**

```
// Use const by default
const PI = 3.14159;
const users = [];

// Use let when reassignment needed
let count = 0;
count++;

// Avoid var
// const and let are block-scoped
```

Comparison

✅ **DO:**

```
// Use strict equality
if (value === 10) {}
if (value !== null) {}

// Explicit type conversion
if (Number(value) === 10) {}
if (Boolean(value)) {}
```

❌ **DON'T:**

```
// Loose equality (type coercion)
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
if (value == 10) {}
```

Error Handling

✅ **DO:**

```
// Always handle errors
try {
    riskyOperation();
} catch (error) {
    console.error(error);
    // Handle appropriately
}

// Validate inputs
function divide(a, b) {
    if (b === 0) {
        throw new Error("Cannot divide by zero");
    }
    return a / b;
}
```

Async Operations

✅ **DO:**

```
// Use async/await for readability
async function fetchData() {
    try {
        let data = await fetch(url);
        return await data.json();
    } catch (error) {
        console.error(error);
    }
}

// Handle promise rejections
promise.catch(error => console.error(error));
```

Performance

✅ **DO:**

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

```
// Cache DOM queries
let button = document.querySelector("button");
button.addEventListener("click", handler);

// Use event delegation
container.addEventListener("click", (e) => {
    if (e.target.matches(".item")) {
        // Handle
    }
});

// Debounce expensive operations
function debounce(func, delay) {
    let timeoutId;
    return function(...args) {
        clearTimeout(timeoutId);
        timeoutId = setTimeout(() => func.apply(this, args), delay);
    };
}
```

Comments

✅ **DO:**

```
// Explain why, not what
// Check if user has premium subscription
// because free users have different limits
if (user.isPremium) {
    // ...
}

// Document complex functions
/**
 * Calculates compound interest
 * @param {number} principal - Initial amount
 * @param {number} rate - Interest rate (decimal)
 * @param {number} years - Number of years
 * @returns {number} Final amount
 */
function calculateInterest(principal, rate, years) {
    return principal * Math.pow(1 + rate, years);
}
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

Confide Computer Academy © COPYRIGHT 2025

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

15. JavaScript Cheat Sheet

Variables

```
const name = "value";     // Cannot reassign
let age = 30;             // Can reassign
// var - avoid using
```

Data Types

String, Number, Boolean, Undefined, Null, Symbol, BigInt
Object, Array, Function

Operators

```
+, -, *, /, %, **        // Arithmetic
=, +=, -=, *=, /=        // Assignment
==, ===, !=, !==         // Comparison
<, >, <=, >=             // Relational
&&, ||, !                // Logical
? :                      // Ternary
```

Functions

```
function name() {}              // Declaration
const name = function() {}      // Expression
const name = () => {}           // Arrow
```

Arrays

```
.push()    .pop()     .shift()    .unshift()
.map()     .filter()  .reduce()   .forEach()
.find()    .includes() .indexOf() .slice()
```

Objects

```
Object.keys()    Object.values()    Object.entries()
Object.assign()  .hasOwnProperty()
```

DOM

```
document.querySelector()
```

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY

document.querySelectorAll()
element.addEventListener()
element.innerHTML / .textContent
element.classList.add/remove/toggle()

Async

setTimeout() setInterval()
Promise async/await
fetch() .then() .catch()

ES6+

Template literals: `${}`
Destructuring: { a, b } = obj
Spread: ...arr
Rest: ...params
Arrow functions: () => {}
Classes: class Name {}

Resources

Documentation:

- MDN JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- JavaScript.info: https://javascript.info

Practice:

- freeCodeCamp
- Codecademy
- LeetCode (algorithms)

Tools:

- Chrome DevTools
- VS Code
- ESLint (linting)

END OF JAVASCRIPT GUIDE

© COPYRIGHT 2025 CONFIDE COMPUTER ACADEMY